UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RAFAELA

- Tecnicatura Universitaria en Programación -

Gestión de Desarrollo de Software



Unidad I: Introducción

Docente: Ing. Darío Haspert

Curso: 2º año - 2º cuatr.

< julio de 2026 >

Índice

UNIDAD 1. Introducción 3
1. Introducción 3
2. La naturaleza del software 3
2.1. Definición de software 4
2.2. Software heredado o “Legacy software” 4
2.3. Productos 5
3. La Ingeniería de Software 6
4. El proceso de Software 8
4.1. Modelos de Proceso 11
5. Proyecto de software vs proceso de desarrollo 18
6. La Ingeniería de requerimientos 18
6.1. Requerimientos funcionales y no funcionales 20
Referencias Bibliográficas 22

UNIDAD 1. Introducción

1. Introducción

El software se ha introducido en casi todos los aspectos de la vida moderna. Esta omnipresencia ha traído consigo una consecuencia directa:

- los sistemas que se construyen son cada vez más complejos,
- involucran a más personas con intereses distintos, y
- deben ser confiables, mantenibles y adaptables a lo largo del tiempo.

Estas exigencias no pueden satisfacerse con intuición o improvisación. Requieren un enfoque disciplinado y sistemático: la *ingeniería de software*.

2. La naturaleza del software

En la actualidad, el **software** tiene un papel dual. Es un **producto** y al mismo tiempo es el **vehículo** para entregar un producto. En su forma de **producto**, brinda el potencial de cómputo incorporado en el hardware de cómputo o, con más amplitud, en una red de computadoras a las que se accede por medio de un hardware local. Ya sea que resida en un teléfono móvil u opere en el interior de una computadora central, el **software** es un transformador de información —produce, administra, adquiere, modifica, despliega o transmite información que puede ser tan simple como un solo bit o tan compleja como una presentación con multimedios generada a partir de datos obtenidos de decenas de fuentes independientes—. Como vehículo utilizado para distribuir el producto, el **software** actúa como la base para el control de la computadora (sistemas operativos), para la comunicación de información (redes) y para la creación y control de otros programas (herramientas y ambientes de software).

El **software** distribuye el producto más importante de nuestro tiempo: *información*. Transforma los datos personales (por ejemplo, las transacciones financieras de un individuo) de modo que puedan ser más útiles en un contexto local, administra la información de negocios para mejorar la competitividad, provee una vía para las redes mundiales de información (la internet) y brinda los medios para obtener información en todas sus formas.

En el último medio siglo, el papel del software de cómputo ha sufrido un cambio significativo. Las notables mejoras en el funcionamiento del hardware, los profundos cambios en las arquitecturas de computadora, el gran incremento en la memoria y capacidad de almacenamiento, y una amplia variedad de opciones de entradas y salidas exóticas han propiciado la existencia de sistemas basados en computadora más sofisticados y complejos. Cuando un sistema tiene éxito, la sofisticación y complejidad producen resultados deslumbrantes, pero también plantean problemas enormes para aquellos que deben construir sistemas complejos.

En la actualidad, la enorme industria del software se ha convertido en un factor dominante en las economías del mundo industrializado. Equipos de especialistas de software, cada uno centrado en una parte de la tecnología que se requiere para llegar a una aplicación compleja, han reemplazado al programador solitario de los primeros tiempos. A pesar de ello, las preguntas que se hacía aquel

programador son las mismas que surgen cuando se construyen sistemas modernos basados en computadora:

- ¿Por qué se requiere tanto tiempo para terminar el software?
- ¿Por qué son tan altos los costos de desarrollo?
- ¿Por qué no podemos detectar todos los errores antes de entregar el software a nuestros clientes?
- ¿Por qué dedicamos tanto tiempo y esfuerzo a mantener los programas existentes?
- ¿Por qué seguimos con dificultades para medir el avance mientras se desarrolla y mantiene el software?

Éstas y muchas otras preguntas, denotan la preocupación sobre el software y la manera en que se desarrolla, preocupación que ha llevado a la adopción de la práctica de la ingeniería del software.

2.1. Definición de software

En la actualidad, la mayoría de los profesionales y muchos usuarios tienen la fuerte sensación de que entienden el software. Pero, ¿es así?

La descripción que daría un libro de texto sobre software sería más o menos así:

El software es: 1) **instrucciones** (programas de cómputo) que cuando se ejecutan proporcionan las características, función y desempeño buscados; 2) **estructuras de datos** que permiten que los programas manipulen en forma adecuada la información, y 3) **información descriptiva** tanto en papel como en formas virtuales que describen la operación y uso de los programas.

No hay duda de que podrían darse definiciones más completas.

Pero es probable que una definición más formal no mejore de manera apreciable nuestra comprensión. Para asimilar lo anterior, es importante examinar las características del software que lo hacen diferente de otros objetos que construyen los seres humanos. El software es elemento de un sistema lógico y no de uno físico. Por tanto, tiene características que difieren considerablemente de las del hardware:

- El software se desarrolla o ingenia, no se fabrica en el sentido clásico.
- El software no se "desgasta", pero sí se deteriora.
- Aunque la industria se está orientando hacia la construcción basada en componentes, la mayor parte del software sigue siendo personalizado.

2.2. Software heredado o “Legacy software”

Se denomina *legacy software* (o legado) a todo sistema de software que fue desarrollado con tecnologías o metodologías anteriores, que sigue en uso activo dentro de una organización, y cuya modificación o reemplazo representa un riesgo o costo significativo.

La característica definitoria no es su antigüedad sino su condición: es un sistema que funciona, que la organización no puede simplemente apagar, pero que fue construido bajo restricciones y decisiones técnicas que hoy resultan difíciles de sostener. Código sin documentación, arquitecturas no escalables,

dependencias de plataformas obsoletas y ausencia de pruebas automatizadas son sus síntomas más frecuentes.

El legacy software es relevante para la gestión de proyectos porque muchos proyectos reales no parten de cero: intervienen, extienden o reemplazan sistemas existentes de este tipo, lo que impone restricciones adicionales de alcance, riesgo y estimación.

2.3. Productos

Los ingenieros de software están interesados por el desarrollo de productos de software (es decir, software que puede venderse a un cliente). Existen dos tipos de productos de software: [3]

1. *Productos genéricos:* Consisten en sistemas independientes que se producen por una organización de desarrollo y se venden en el mercado abierto a cualquier cliente que desee comprarlos. Ejemplos de este tipo de productos incluyen software para PC, como bases de datos, procesadores de texto, paquetes de dibujo y herramientas de administración de proyectos. También abarcan las llamadas aplicaciones verticales diseñadas para cierto propósito específico, tales como sistemas de información de librería, sistemas de contabilidad o sistemas para mantener registros dentales.

2. *Productos personalizados (o a la medida):* Son sistemas que están destinados para un cliente en particular. Un contratista de software desarrolla el programa especialmente para dicho cliente. Ejemplos de este tipo de software incluyen los sistemas de control para dispositivos electrónicos, sistemas escritos para apoyar cierto proceso empresarial y los sistemas de control de tráfico aéreo.

Una diferencia importante entre estos tipos de software es que, en productos genéricos, la organización que desarrolla el software controla la especificación del mismo. Para los productos personalizados, la organización que compra el software generalmente desarrolla y controla la especificación, por lo que los desarrolladores de software deben trabajar siguiendo dicha especificación.

Sin embargo, la distinción entre estos tipos de producto de sistemas se vuelve cada vez más difusa. Ahora, cada vez más sistemas se construyen con un producto genérico como base, que luego se adapta para ajustarse a los requerimientos de un cliente. Los sistemas Enterprise Resource Planning (ERP, planeación de recursos empresariales), como el sistema SAP, son los mejores ejemplos de este enfoque. Aquí, un sistema grande y complejo se adapta a una compañía al incorporar la información acerca de las reglas y los procesos empresariales, los reportes requeridos, etcétera.

Cuando se habla de la calidad del software profesional, se debe considerar que el software lo usan y cambian personas, además de sus desarrolladores. En consecuencia, la calidad no tiene que ver sólo con lo que hace el software. En cambio, debe incluir el comportamiento del software mientras se ejecuta, y la estructura y organización de los programas del sistema y la documentación asociada. Esto se refleja en los llamados atributos de calidad o no funcionales del software. Ejemplos de dichos atributos son el tiempo de respuesta del software ante la duda de un usuario y la comprensibilidad del código del programa.

El conjunto específico de atributos que se espera de un sistema de software depende evidentemente de su aplicación. Así, un sistema bancario debe ser seguro, un juego interactivo debe tener capacidad de respuesta, un sistema de conmutación telefónica debe ser confiable, etcétera.

3. La Ingeniería de Software

Investigar más sobre esta sección y dejarla para el final

El IEEE ha desarrollado la siguiente definición:

La **ingeniería de software** es:

1) La aplicación de un enfoque sistemático, disciplinado y cuantificable al desarrollo, operación y mantenimiento de software; es decir, la aplicación de la ingeniería al software.
2) El estudio de enfoques según el punto 1.

Aun así, el enfoque "sistemático, disciplinado y cuantificable" aplicado por un equipo de software podría ser algo burdo para otro. Se necesita disciplina, pero también adaptabilidad y agilidad.

La **ingeniería de software** es una tecnología con varias capas. Como se aprecia en la figura siguiente, cualquier enfoque de ingeniería (incluso la de software) debe basarse en un compromiso organizacional con la calidad. La administración total de la calidad (TQM), Six Sigma y otras filosofías similares alimentan la cultura de mejora continua, y es esta cultura la que lleva en última instancia al desarrollo de enfoques cada vez más eficaces de la ingeniería de software. La base en la que se apoya la ingeniería de software es el ***compromiso con la calidad***.



Capas de la ingeniería de software

El fundamento para la ingeniería de software es la capa ***proceso***. El proceso de ingeniería de software es el aglutinante que une las capas de la tecnología y permite el desarrollo racional y oportuno del software de computadoras. El proceso define una estructura que debe establecerse para la obtención eficaz de tecnología de ingeniería de software. El proceso de software forma la base para el control de la administración de proyectos de software, y establece el contexto en el que se aplican métodos técnicos, se generan productos del trabajo (modelos, documentos, datos, reportes, formatos, etc.), se establecen puntos de referencia, se asegura la calidad y se administra el cambio de manera apropiada.

Los ***métodos*** de la ingeniería de software proporcionan la experiencia técnica para elaborar software. Incluyen un conjunto amplio de tareas, como comunicación, análisis de los requerimientos, modelación del diseño, construcción del programa, pruebas y apoyo. Los métodos de la ingeniería de software se basan en un conjunto de principios fundamentales que gobiernan cada área de la tecnología e incluyen actividades de modelación y otras técnicas descriptivas.

Las ***herramientas*** de la ingeniería de software proporcionan un apoyo automatizado o semi-automatizado para el proceso y los métodos. Cuando se integran las herramientas de modo que la información creada por una pueda ser utilizada por otra, queda establecido un sistema llamado *ingeniería de software asistido por computadora* que apoya el desarrollo de software.

Para obtener una amplia visión de lo que trata la ingeniería de software, en la siguiente tabla se resumen algunas preguntas planteadas con frecuencia. [3]

Pregunta	**Respuesta**
¿Qué es software?	Programas de cómputo y documentación asociada. Los productos de software se desarrollan para un cliente en particular o para un mercado en general.
¿Cuáles son los atributos del buen software?	El buen software debe entregar al usuario la funcionalidad y el desempeño requeridos, y debe ser sustentable, confiable y utilizable.
¿Qué es ingeniería de software?	La ingeniería de software es una disciplina de la ingeniería que se interesa por todos los aspectos de la producción de software.
¿Cuáles son las actividades fundamentales de la ingeniería de software?	Especificación, desarrollo, validación y evolución del software.
¿Cuál es la diferencia entre ingeniería de software y ciencias de la computación?	Las ciencias de la computación se enfocan en teoría y fundamentos; mientras la ingeniería de software se enfoca en el sentido práctico del desarrollo y en la distribución de software.
¿Cuál es la diferencia entre ingeniería de software e ingeniería de sistemas?	La ingeniería de sistemas se interesa por todos los aspectos del desarrollo de sistemas basados en computadoras, incluidos hardware, software e ingeniería de procesos. La ingeniería de software es parte de este proceso más general.
¿Cuáles son los principales retos que enfrenta la ingeniería de software?	Se enfrentan con una diversidad creciente, demandas por tiempos de distribución limitados y desarrollo de software confiable.
¿Cuáles son los costos de la ingeniería de software?	Aproximadamente 60% de los costos del software son de desarrollo, y 40% de prueba. Para el software elaborado específicamente, los costos de evolución superan con frecuencia los costos de desarrollo.
¿Cuáles son los mejores métodos y técnicas de la ingeniería de software?	Aun cuando todos los proyectos de software deben gestionarse y desarrollarse de manera profesional, existen diferentes técnicas que son adecuadas para distintos tipos de sistema. Por ejemplo, los juegos siempre deben diseñarse usando una serie de prototipos, mientras que los sistemas críticos de control de seguridad requieren de una especificación completa y analizable para su desarrollo. Por lo tanto, no puede decirse que un método sea mejor que otro.

4. El proceso de Software

Un **proceso** es un conjunto de actividades, acciones y tareas que se ejecutan cuando va a crearse algún producto del trabajo. Una *actividad* busca lograr un objetivo amplio (por ejemplo, comunicación con los participantes) y se desarrolla sin importar el dominio de la aplicación, tamaño del proyecto, complejidad del esfuerzo o grado de rigor con el que se usará la ingeniería de software. Una *acción* (diseño de la arquitectura) es un conjunto de tareas que producen un producto importante del trabajo (por ejemplo, un modelo del diseño de la arquitectura). Una *tarea* se centra en un objetivo pequeño pero bien definido (por ejemplo, realizar una prueba unitaria) que produce un resultado tangible.

En el contexto de la ingeniería de software, un proceso no es una prescripción rígida de cómo elaborar software de cómputo. Por el contrario, es un enfoque adaptable que permite que las personas que hacen el trabajo (el equipo de software) busquen y elijan el conjunto apropiado de acciones y tareas para el trabajo. Se busca siempre entregar el software en forma oportuna y con calidad suficiente para satisfacer a quienes patrocinaron su creación y a aquellos que lo usarán.

La *estructura del proceso* establece el fundamento para el proceso completo de la ingeniería de software por medio de la identificación de un número pequeño de *actividades estructurales* que sean aplicables a todos los proyectos de software, sin importar su tamaño o complejidad. Además, la estructura del proceso incluye un conjunto de *actividades sombrilla* que son aplicables a través de todo el proceso del software. Una estructura de proceso general para la ingeniería de software consta de cinco actividades:

1. **Comunicación.** Antes de que comience cualquier trabajo técnico, tiene importancia crítica comunicarse y colaborar con el cliente (y con otros participantes). Se busca entender los objetivos de los participantes respecto del proyecto, y reunir los requerimientos que ayuden a definir las características y funciones del software.
2. **Planeación.** Cualquier viaje complicado se simplifica si existe un mapa. Un proyecto de software es un viaje difícil, y la actividad de planeación crea un "mapa" que guía al equipo mientras viaja. El mapa —llamado *plan del proyecto de software*— define el trabajo de ingeniería de software al describir las tareas técnicas por realizar, los riesgos probables, los recursos que se requieren, los productos del trabajo que se obtendrán y una programación de las actividades.
3. **Modelado.** Ya sea usted diseñador de paisaje, constructor de puentes, ingeniero aeronáutico, carpintero o arquitecto, a diario trabaja con modelos. Crea un "bosquejo" del objeto por hacer a fin de entender el panorama general —cómo se verá arquitectónicamente, cómo ajustan entre sí las partes constituyentes y muchas características más—. Si se requiere, refina el bosquejo con más y más detalles en un esfuerzo por comprender mejor el problema y cómo resolverlo. Un ingeniero de software hace lo mismo al crear modelos a fin de entender mejor los requerimientos del software y el diseño que los satisfará.
4. **Construcción.** Esta actividad combina la generación de código (ya sea manual o automatizada) y las pruebas que se requieren para descubrir errores en éste.
5. **Despliegue.** El software (como entidad completa o como un incremento parcialmente terminado) se entrega al consumidor que lo evalúa y que le da retroalimentación, misma que se basa en dicha evaluación.

Estas cinco actividades estructurales genéricas se usan durante el desarrollo de programas pequeños y sencillos, en la creación de aplicaciones web grandes y en la ingeniería de sistemas enormes y complejos

basados en computadoras. Los detalles del proceso de software serán distintos en cada caso, pero las actividades estructurales son las mismas.

Para muchos proyectos de software, las actividades estructurales se aplican en forma iterativa a medida que avanza el proyecto. Es decir, la **comunicación,** la **planeación,** el **modelado,** la **construcción** y el **despliegue** se ejecutan a través de cierto número de repeticiones del proyecto. Cada iteración produce un *incremento del software* que da a los participantes un subconjunto de características y funcionalidad generales del software. Conforme se produce cada incremento, el software se hace más y más completo.

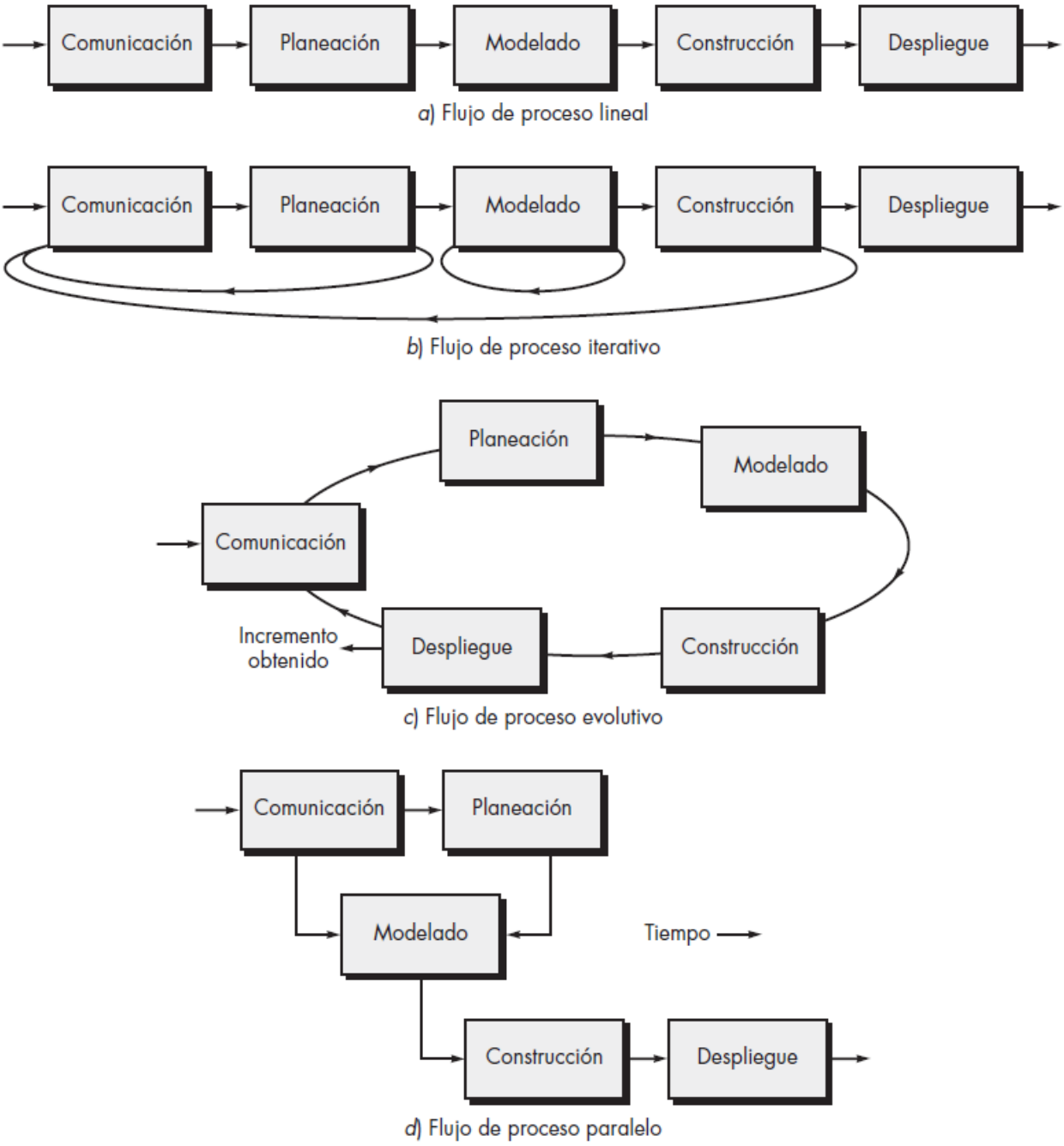
Las actividades estructurales del proceso de ingeniería de software son complementadas por cierto número de actividades sombrilla. En general, las actividades sombrilla se aplican a lo largo de un proyecto de software y ayudan al equipo que lo lleva a cabo a administrar y controlar el avance, la calidad, el cambio y el riesgo. Es común que las actividades sombrilla sean las siguientes:

- **Seguimiento y control del proyecto de software**: permite que el equipo de software evalúe el progreso comparándolo con el plan del proyecto y tome cualquier acción necesaria para apegarse a la programación de actividades.
- **Administración del riesgo**: evalúa los riesgos que puedan afectar el resultado del proyecto o la calidad del producto.
- **Aseguramiento de la calidad del software:** define y ejecuta las actividades requeridas para garantizar la calidad del software.
- **Revisiones técnicas:** evalúa los productos del trabajo de la ingeniería de software a fin de descubrir y eliminar errores antes de que se propaguen a la siguiente actividad.
- **Medición:** define y reúne mediciones del proceso, proyecto y producto para ayudar al equipo a entregar el software que satisfaga las necesidades de los participantes; puede usarse junto con todas las demás actividades estructurales y sombrilla.
- **Administración de la configuración del software:** administra los efectos del cambio a lo largo del proceso del software.
- **Administración de la reutilización:** define criterios para volver a usar el producto del trabajo (incluso los componentes del software) y establece mecanismos para obtener componentes reutilizables.
- **Preparación y producción del producto del trabajo:** agrupa las actividades requeridas para crear productos del trabajo, tales como modelos, documentos, registros, formatos y listas.

Como ya se dijo, el proceso de ingeniería de software no es una prescripción rígida que deba seguir en forma dogmática el equipo que lo crea. Al contrario, debe ser ágil y adaptable (al problema, al proyecto, al equipo y a la cultura organizacional). Por tanto, un proceso adoptado para un proyecto puede ser significativamente distinto de otro adoptado para otro proyecto.

Aún no se ha mencionado un aspecto importante del proceso del software, llamado *flujo del proceso*, que describe la manera en que están organizadas las actividades estructurales y las acciones y tareas que ocurren dentro de cada una con respecto de la secuencia y el tiempo.

Un *flujo de proceso lineal* ejecuta cada una de las cinco actividades estructurales en secuencia, comenzando por la comunicación y terminando con el despliegue (véase la parte *a* de la figura siguiente). Un *flujo de proceso iterativo* repite una o más de las actividades antes de pasar a la siguiente (parte *b* de la figura). Un *flujo de proceso evolutivo* realiza las actividades en forma "circular". A través de las cinco actividades, cada circuito lleva a una versión más completa del software (parte *c* de la figura). Un *flujo de proceso paralelo* (parte *d* de la figura) ejecuta una o más actividades en paralelo con otras (por ejemplo, el modelado de un aspecto del software tal vez se ejecute en paralelo con la construcción de otro aspecto del software).



Flujo del proceso

En ingeniería de software, distinguir entre actividad, acción y tarea permite comprender mejor la estructura del proceso de desarrollo.

Nivel	Definición	Propósito	Ejemplo
Actividad	Conjunto amplio de trabajo orientado a un objetivo general del proceso.	Organizar una etapa importante del desarrollo.	Comunicación con el cliente.
Acción	Conjunto de tareas relacionadas que producen un resultado intermedio relevante.	Descomponer una actividad en partes más concretas.	Modelado de la arquitectura.
Tarea	Unidad pequeña, puntual y bien definida de trabajo.	Ejecutar una acción específica.	Realizar una prueba unitaria.

4.1. Modelos de Proceso

Apoyarme de los resumenes ya echos

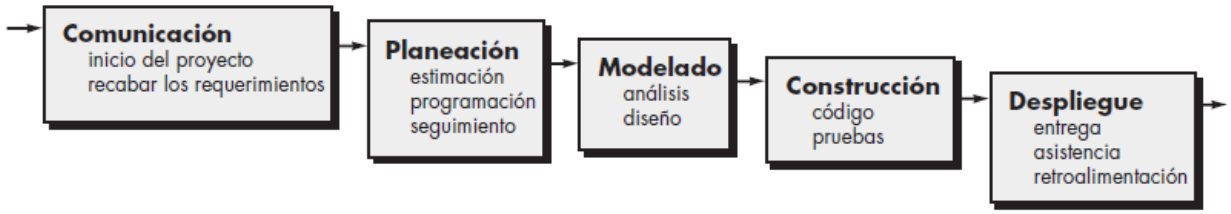
Un modelo de proceso define cómo se organizan y secuencian las actividades del desarrollo de software. No existe un modelo universalmente correcto: cada uno representa un conjunto de compromisos entre orden, flexibilidad, visibilidad temprana del producto y gestión del riesgo.

Los modelos de procesos prescriptivos definen un conjunto preestablecido de elementos de proceso y un flujo de trabajo predecible. Estos modelos buscan aportar estructura y orden al desarrollo de software. Las actividades y tareas se llevan a cabo de forma secuencial, siguiendo unas directrices definidas para su avance.

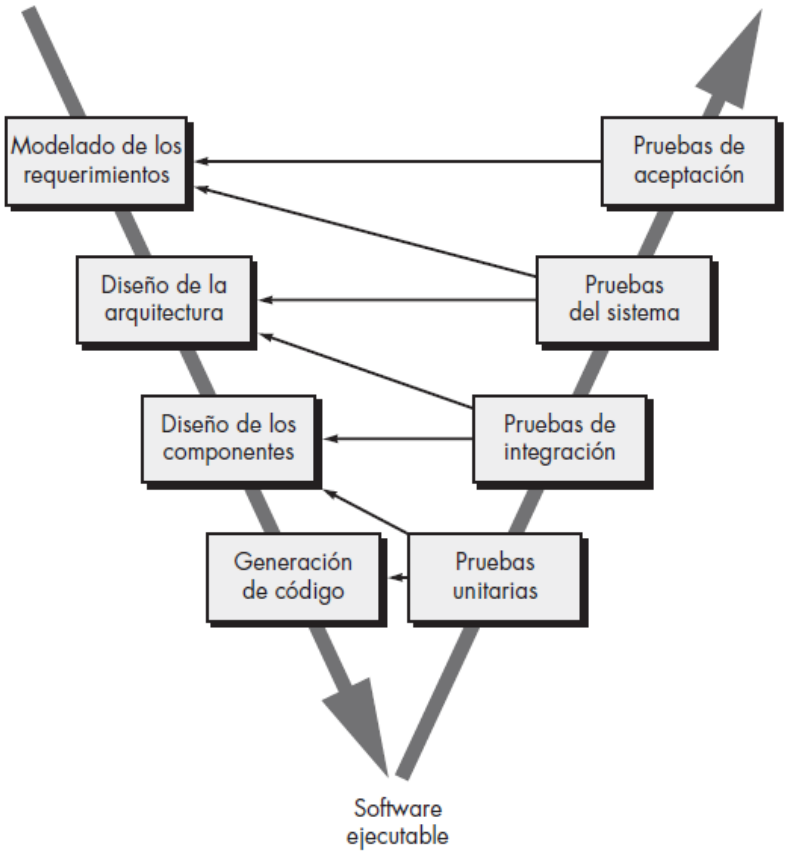
En el enfoque de proceso prescriptivo, el orden y la coherencia del proyecto son cuestiones dominantes. Los llamamos «prescriptivos» porque prescriben un conjunto de elementos de proceso: actividades estructurales, acciones de ingeniería de software, tareas, productos de trabajo, aseguramiento de calidad y mecanismos de control de cambios para cada proyecto. Cada modelo de proceso también prescribe un flujo de proceso (o *flujo de trabajo*), es decir, la manera en que los elementos del proceso se interrelacionan entre sí.

Modelo de la Cascada

El *modelo de la cascada*, a veces llamado *ciclo de vida clásico*, sugiere un enfoque sistemático y secuencial para el desarrollo del software, que comienza con la especificación de los requerimientos por parte del cliente y avanza a través de planeación, modelado, construcción y despliegue, para concluir con el apoyo del software terminado. Cada fase debe completarse antes de iniciar la siguiente. Es predecible y fácil de planificar, pero rígido: cualquier cambio en los requerimientos después de iniciado el desarrollo tiene un costo alto.



Una variante de la representación del modelo de la cascada se denomina *modelo en V*. En la siguiente figura se ilustra el modelo en V, donde se aprecia la relación entre las acciones para el aseguramiento de la calidad y aquellas asociadas con la comunicación, modelado y construcción temprana. A medida que el equipo de software avanza hacia abajo desde el lado izquierdo de la V, los requerimientos básicos del problema mejoran hacia representaciones técnicas cada vez más detalladas del problema y de su solución. Una vez que se ha generado el código, el equipo sube por el lado derecho de la V, y en esencia ejecuta una serie de pruebas (acciones para asegurar la calidad) que validan cada uno de los modelos creados cuando el equipo fue hacia abajo por el lado izquierdo. En realidad, no hay diferencias fundamentales entre el ciclo de vida clásico y el modelo en V. Este último proporciona una forma de visualizar el modo de aplicación de las acciones de verificación y validación al trabajo de ingeniería inicial.



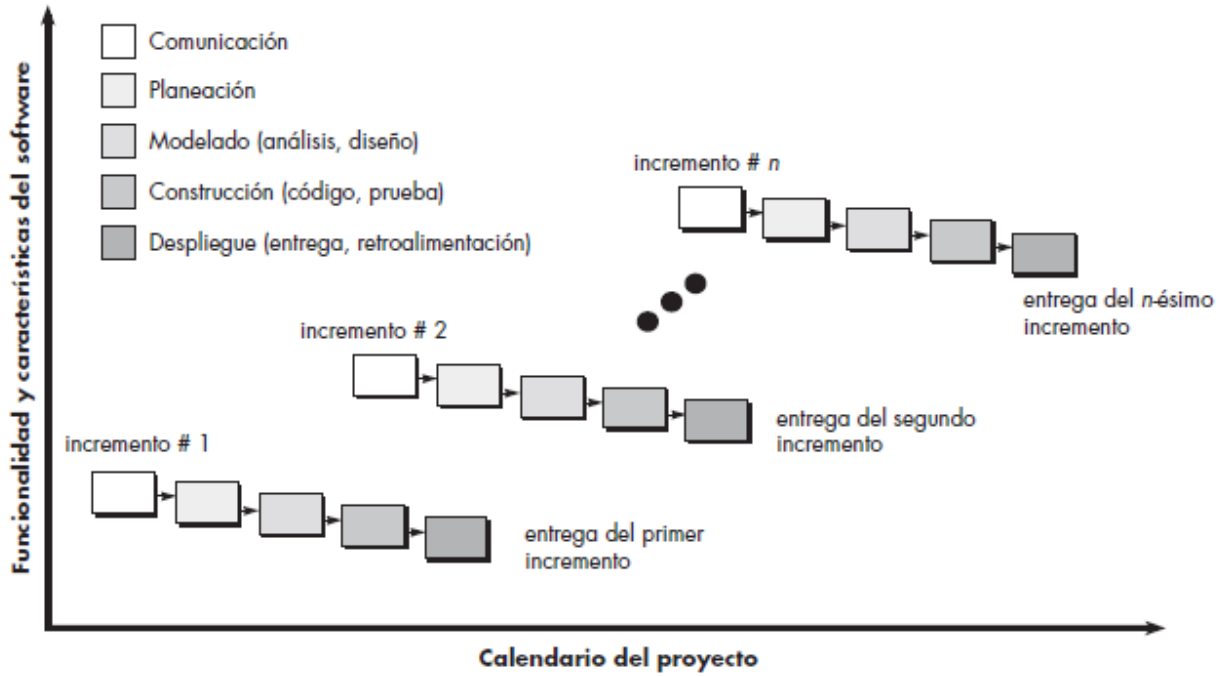
El modelo de la cascada es el paradigma más antiguo de la ingeniería de software. Sin embargo, en las últimas décadas, las críticas hechas al modelo han ocasionado que incluso sus defensores más obstinados cuestionen su eficacia.

Hoy en día, el trabajo de software es acelerado y está sujeto a una corriente sin fin de cambios (en las características, funciones y contenido de información). El modelo de la cascada suele ser inapropiado para ese tipo de labor. No obstante, sirve como un modelo de proceso útil en situaciones en las que los requerimientos son fijos y el trabajo avanza en forma lineal hacia el final.

Modelos de proceso incremental

El modelo *incremental* combina elementos de los flujos de proceso lineal y paralelo. En relación con la figura siguiente, el modelo incremental aplica secuencias lineales en forma escalonada a medida que avanza el calendario de actividades. Cada secuencia lineal produce "incrementos" de software susceptibles de entregarse de manera parecida a los incrementos producidos en un flujo de proceso evolutivo.

Cuando se utiliza un modelo incremental, es frecuente que el primer incremento sea el *producto fundamental*. Es decir, se abordan los requerimientos básicos, pero no se proporcionan muchas características suplementarias (algunas conocidas y otras no). El cliente usa el producto fundamental (o lo somete a una evaluación detallada). Como resultado del uso y/o evaluación, se desarrolla un plan para el incremento que sigue. El plan incluye la modificación del producto fundamental para cumplir mejor las necesidades del cliente, así como la entrega de características adicionales y más funcionalidad. Este proceso se repite después de entregar cada incremento, hasta terminar el producto final.

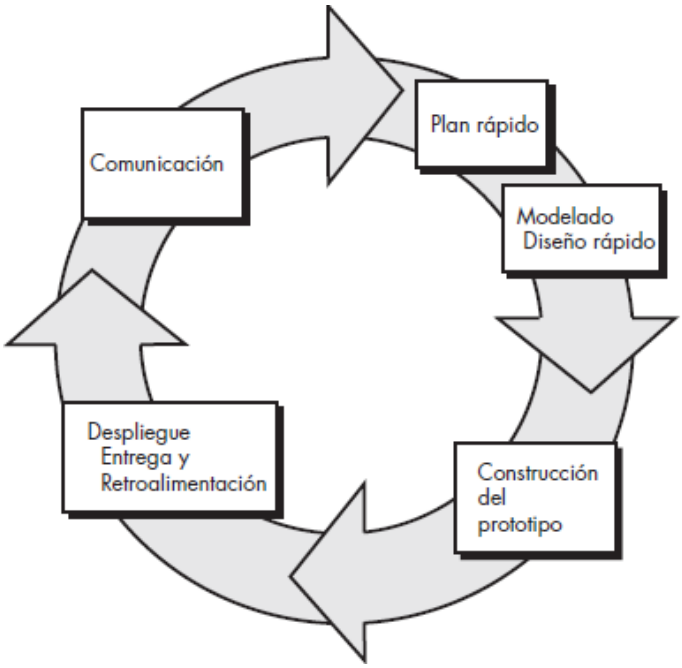


El modelo de proceso incremental se centra en que en cada incremento se entrega un producto que ya opera.

Modelos de proceso evolutivo

Son modelos de proceso diseñados explícitamente para adaptarse a un producto que evoluciona con el tiempo. Los modelos evolutivos son iterativos. Se caracterizan por la manera en la que permiten desarrollar versiones cada vez más completas del software.

Hacer prototipos: El paradigma de hacer prototipos comienza con comunicación. Se definen los objetivos generales del software, se identifican requerimientos y detectan las áreas en las que es imprescindible una mayor definición. Se planea rápidamente una iteración para hacer el prototipo, y se lleva a cabo el modelado (en forma de un "diseño rápido"). Éste se centra en la representación de aquellos aspectos del software que serán visibles para los usuarios finales (por ejemplo, disposición de la interfaz humana o formatos de la pantalla de salida). El diseño rápido lleva a la construcción de un prototipo. Éste se entrega y es evaluado por los participantes, que dan retroalimentación para mejorar los requerimientos. La iteración ocurre a medida de que el prototipo es afinado para satisfacer las necesidades de distintos participantes, y al mismo tiempo le permite a usted entender mejor lo que se necesita hacer.

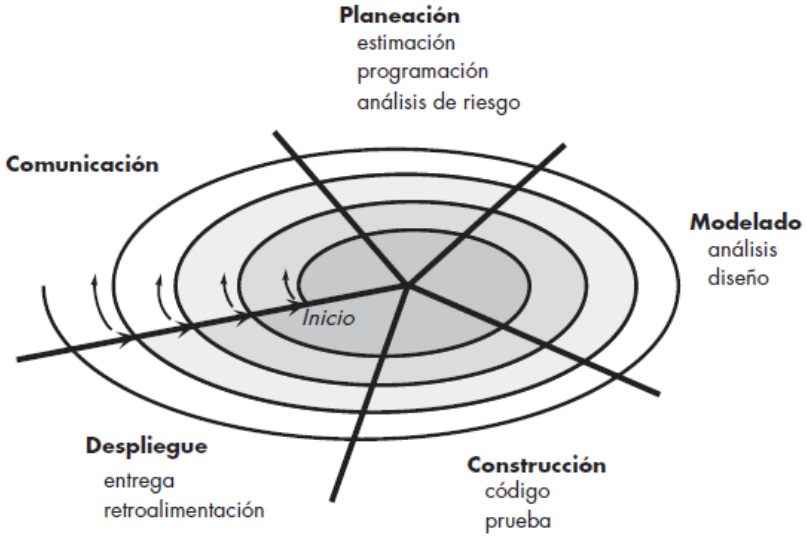


El modelo espiral: Es un modelo evolutivo del proceso del software y se acopla con la naturaleza iterativa de hacer prototipos con los aspectos controlados y sistémicos del modelo de cascada. Tiene el potencial para hacer un desarrollo rápido de versiones cada vez más completas.

Con el empleo del modelo espiral, el software se desarrolla en una serie de entregas evolutivas. Durante las primeras iteraciones, lo que se entrega puede ser un modelo o prototipo. En las iteraciones posteriores se producen versiones cada vez más completas del sistema cuya ingeniería se está haciendo.

El modelo espiral es un enfoque realista para el desarrollo de sistemas y de software a gran escala. Como el software evoluciona a medida que el proceso avanza, el desarrollador y cliente comprenden y reaccionan mejor ante los riesgos en cada nivel de evolución. El modelo espiral usa los prototipos como mecanismo de reducción de riesgos y permite aplicar el enfoque de hacer prototipos en cualquier etapa de la evolución del producto. Mantiene el enfoque de escalón sistemático sugerido por el ciclo de vida clásico, pero lo incorpora en una estructura iterativa que refleja al mundo real en una forma más realista. El modelo espiral demanda una consideración directa de los riesgos técnicos en todas las etapas del proyecto y, si se aplica de manera apropiada, debe reducir los riesgos antes de que se vuelvan un problema.

Es difícil convencer a los clientes de que el enfoque evolutivo es controlable. Demanda mucha experiencia en la evaluación del riesgo y se basa en ella para llegar al éxito. No hay duda de que habrá problemas si algún riesgo importante no se descubre y administra.

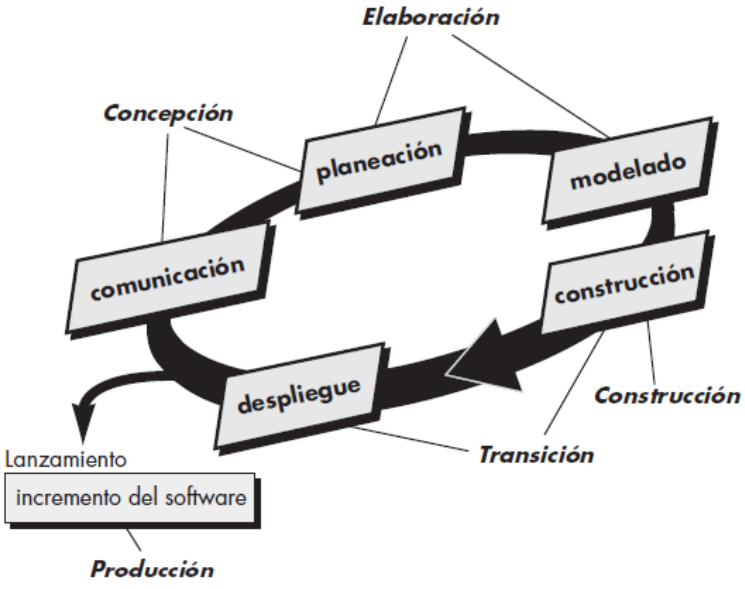


El Modelo de Proceso Unificado

En cierto modo, el proceso unificado es un intento por obtener los mejores rasgos y características de los modelos tradicionales del proceso del software, pero de forma que implemente muchos de los mejores principios del desarrollo ágil de software. El proceso unificado reconoce la importancia de la comunicación con el cliente y los métodos directos para describir su punto de vista respecto de un sistema (el caso de uso). Hace énfasis en la importancia de la arquitectura del software y "ayuda a que el arquitecto se centre en las metas correctas, tales como que sea comprensible, permita cambios futuros y la reutilización". Sugiere un flujo del proceso iterativo e incremental, lo que da la sensación evolutiva que resulta esencial en el desarrollo moderno del software.

El UML (Lenguaje Unificado de Modelado) se desarrolló para facilitar este trabajo. El UML contiene una notación sólida para el modelado y el desarrollo de sistemas orientados a objetos y se ha convertido en un estándar de facto del sector para el modelado de todo tipo de software.

La siguiente figura ilustra las 5 "fases" del PU y las relaciona con las actividades generales descritas previamente.



La *fase de Concepción* (Inception) del PU es donde tienen lugar la comunicación con el cliente y la planificación. Los requisitos empresariales fundamentales se describen mediante un conjunto de casos de uso preliminares que detallan qué características y funciones desea cada clase principal de usuarios y que se materializarán en la arquitectura del software. La planificación identifica los recursos, evalúa los principales riesgos y define un calendario preliminar para los incrementos del software.

La *fase de elaboración* abarca las actividades de planificación y modelado del modelo de proceso genérico. La elaboración refina y amplía los casos de uso preliminares y crea una línea de base arquitectónica que incluye cinco vistas diferentes del software: el modelo de casos de uso, el modelo de análisis, el modelo de diseño, el modelo de implementación y el modelo de despliegue. A menudo se realizan modificaciones en el plan en este momento.

La *fase de construcción* del PU es idéntica a la actividad de construcción definida para el proceso de software genérico. A continuación, todas las características y funciones necesarias y requeridas para el incremento de software (es decir, la versión) se implementan en código fuente. A medida que se implementan los componentes, se diseñan y ejecutan pruebas unitarias para cada uno de ellos. Además, se llevan a cabo actividades de integración (ensamblaje de componentes y pruebas de integración). Los casos de uso se utilizan para derivar un conjunto de pruebas de aceptación que se ejecutan antes del inicio de la siguiente fase del PU.

La *fase de transición* del PU abarca las últimas etapas de la actividad genérica de construcción y la primera parte de la actividad genérica de implementación (entrega y retroalimentación). El software y la documentación de apoyo se entregan a los usuarios finales para que realicen pruebas beta, y los comentarios de los usuarios informan tanto de los defectos como de los cambios necesarios. Al concluir la fase de transición, el incremento de software se convierte en una versión de software utilizable.

La *fase de producción* del PU coincide con la actividad de implementación del proceso genérico. Durante esta fase, se supervisa el uso continuo del software, se proporciona soporte para el entorno operativo (infraestructura) y se envían y evalúan los informes de defectos y las solicitudes de cambios.

Es probable que, al mismo tiempo que se llevan a cabo las fases de construcción, transición y producción, ya se haya comenzado a trabajar en el siguiente incremento de software. Esto significa que las cinco fases del PU no se producen en secuencia, sino con una concurrencia escalonada.

Cabe señalar que no todas las tareas identificadas para un flujo de trabajo del PU se llevan a cabo en todos los proyectos de software. El equipo adapta el proceso (acciones, tareas, subtareas y productos de trabajo) para satisfacer sus necesidades.

En la tabla siguiente se resumen algunos de los puntos fuertes y débiles de los modelos de proceso que hemos analizado. La realidad es que ningún proceso es perfecto para todos los proyectos. Por lo general, el equipo de desarrollo adapta uno o varios de los modelos de proceso para satisfacer las necesidades del proyecto en cuestión.

Si el proceso es deficiente, el producto final se verá sin duda afectado. Pero una dependencia excesiva y obsesiva del proceso también es peligrosa.

Modelo	📌 Idea central	✔ Ventajas	❌ Desventajas	🎯 Cuándo usarlo
Cascada	El desarrollo avanza de forma secuencial, etapa por etapa.	Fácil de entender y planificar; útil para proyectos pequeños y bien definidos; análisis y pruebas sencillos.	Poco flexible ante cambios; las pruebas llegan tarde; el cliente ve el producto casi al final.	Cuando los requerimientos son estables y bien conocidos.
Prototipado	Se construye una versión temprana para aclarar requerimientos.	Permite responder mejor a cambios; favorece la participación del cliente; reduce el riesgo de rechazo del producto.	Puede generar retrasos; el prototipo puede confundirse con el producto final; se desperdicia trabajo si se descarta.	Cuando hay incertidumbre en los requerimientos o se necesita validar la interfaz.
Incremental	El sistema se entrega en partes funcionales sucesivas.	Entrega temprana de una versión operativa; permite priorizar funciones; facilita retroalimentación continua.	Requiere buena planificación de incrementos; puede generar arquitectura débil si no se diseña bien.	Cuando se quiere entregar valor por etapas y ampliar el sistema gradualmente.
Espiral	Combina iteración con análisis explícito de riesgos.	Gestión formal de riesgos; adecuado para proyectos grandes y complejos; permite evolucionar el sistema con control.	Gestión compleja; requiere experiencia; el análisis de riesgos puede fallar si se hace mal.	Cuando el proyecto es grande, crítico o presenta alta incertidumbre.
Proceso unificado	Iterativo e incremental, centrado en casos de uso y arquitectura.	Buen equilibrio entre documentación y adaptación; se ajusta a cambios; útil en proyectos de mantenimiento y evolución.	Puede ser complejo; la superposición de fases puede confundir; requiere experiencia del equipo.	Cuando se necesita iterar con orden y mantener foco arquitectónico.

5. Proyecto de software vs proceso de desarrollo

Luego de analizar el proceso de software y los distintos modelos que lo representan, resulta necesario distinguir entre el proceso de desarrollo y el proyecto de software en el cual dicho proceso se aplica.

En el ámbito de la Ingeniería de Software, es habitual que se utilicen de forma indistinta los términos *proyecto de software* y *proceso de desarrollo*, aunque conceptualmente refieren a **entidades diferentes**, con objetivos, alcances y duraciones distintas. La distinción entre ambos conceptos es fundamental para comprender adecuadamente la organización del trabajo en el desarrollo de sistemas de software.

El **proceso de desarrollo de software** puede definirse como el **conjunto estructurado de actividades, métodos, prácticas y herramientas** utilizadas para producir, validar, mantener y evolucionar software. Este proceso establece *qué actividades se realizan*, *en qué orden* y *con qué objetivos*, independientemente de un producto o proyecto particular. Según Sommerville, un proceso de software define tareas tales como la especificación de requerimientos, el desarrollo, la validación y la evolución del software, constituyendo un marco general aplicable a múltiples productos y contextos.

Por su parte, un **proyecto de software** es una **instancia concreta y temporal** en la que se aplica un proceso de desarrollo para construir o modificar un producto específico. Todo proyecto posee un inicio y un fin definidos, objetivos particulares, restricciones de tiempo, costo y recursos, y produce uno o más entregables claramente identificables. Pressman señala que un proyecto de software implica la planificación, organización y control de actividades orientadas a cumplir un conjunto definido de requisitos dentro de un marco de restricciones.

En este sentido, puede afirmarse que:

- el **proceso** es genérico, reusable y de largo plazo,
- mientras que el **proyecto** es específico, acotado en el tiempo y orientado a un resultado particular.

Un mismo proceso de desarrollo puede ser utilizado en múltiples proyectos, y distintos proyectos dentro de una organización suelen compartir un proceso común, adaptado según el contexto. Sin embargo, cada proyecto presenta características propias que requieren decisiones particulares en cuanto a planificación, estimación, asignación de recursos y gestión de riesgos.

La confusión entre estos conceptos puede conducir a errores frecuentes, como suponer que la mera adopción de un proceso o metodología garantiza el éxito del proyecto, o bien que los problemas de un proyecto se resuelven únicamente modificando aspectos técnicos del proceso. La correcta comprensión de esta diferencia permite abordar el desarrollo de software desde una perspectiva más integral, reconociendo que la calidad del producto final depende tanto de un proceso bien definido como de una adecuada gestión del proyecto en el que dicho proceso se aplica.

6. La Ingeniería de requerimientos

Los requerimientos para un sistema son descripciones de lo que el sistema debe hacer: el servicio que ofrece y las restricciones en su operación. Tales requerimientos reflejan las necesidades de los clientes

por un sistema que atienda cierto propósito, como sería controlar un dispositivo, colocar un pedido o buscar información. Al proceso de descubrir, analizar, documentar y verificar estos servicios y restricciones se le llama *ingeniería de requerimientos* (IR).

La *ingeniería de requerimientos* consiste en el proceso de comprender y definir qué servicios se requieren del sistema, así como la identificación de las restricciones sobre la operación y el desarrollo del sistema. La ingeniería de requerimientos es una etapa particularmente crítica del proceso de software, ya que los errores en esta etapa conducen de manera inevitable a problemas posteriores tanto en el diseño como en la implementación del sistema. [3]

El proceso de *ingeniería de requerimientos* se enfoca en producir un documento de requerimientos convenido que especifique los requerimientos de los interesados que cumplirá el sistema. Por lo general, los requerimientos se presentan en dos niveles de detalle. Los usuarios finales y clientes necesitan un informe de requerimientos de alto nivel; los desarrolladores de sistemas precisan una descripción más detallada del sistema.

Sommerville, define cuatro actividades principales en el proceso de ingeniería de requerimientos:

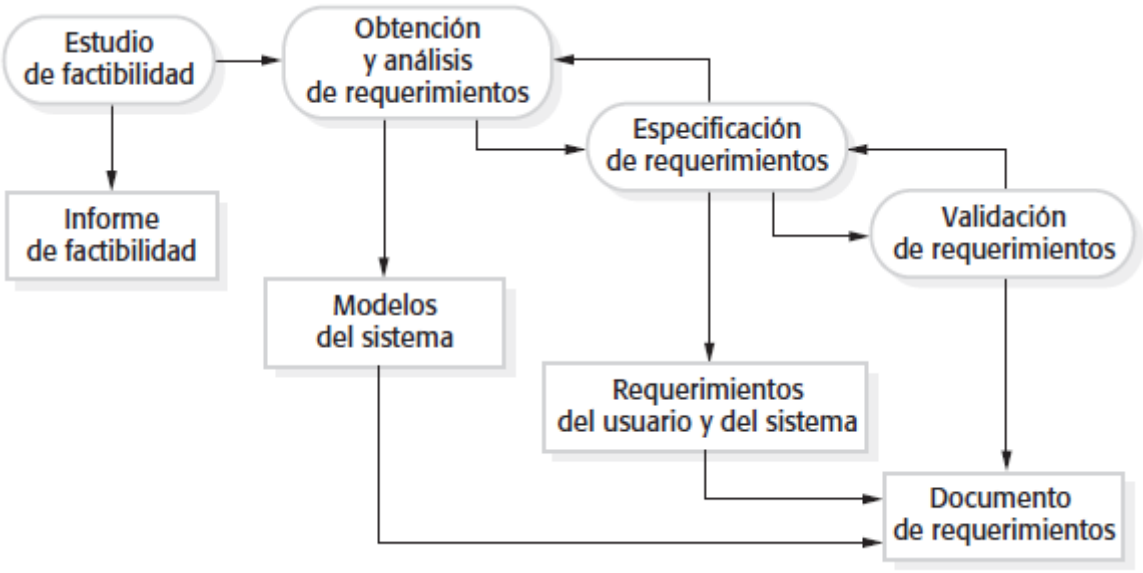
1. *Estudio de factibilidad* Se realiza una estimación sobre si las necesidades identificadas del usuario se cubren con las actuales tecnologías de software y hardware. El estudio considera si el sistema propuesto tendrá un costo-beneficio desde un punto de vista empresarial, y si éste puede desarrollarse dentro de las restricciones presupuestales existentes. Un estudio de factibilidad debe ser rápido y relativamente barato. El resultado debe informar la decisión respecto a si se continúa o no continúa con un análisis más detallado.

2. *Obtención y análisis de requerimientos* Éste es el proceso de derivar los requerimientos del sistema mediante observación de los sistemas existentes, discusiones con los usuarios y proveedores potenciales, análisis de tareas, etcétera. Esto puede incluir el desarrollo de uno o más modelos de sistemas y prototipos, lo que ayuda a entender el sistema que se va a especificar.

3. *Especificación de requerimientos* Consiste en la actividad de transcribir la información recopilada durante la actividad de análisis, en un documento que define un conjunto de requerimientos. En este documento se incluyen dos clases de requerimientos. Los requerimientos del usuario son informes abstractos de requerimientos del sistema para el cliente y el usuario final del sistema; y los requerimientos de sistema son una descripción detallada de la funcionalidad a ofrecer.

4. *Validación de requerimientos* Esta actividad verifica que los requerimientos sean realistas, coherentes y completos. Durante este proceso es inevitable descubrir errores en el documento de requerimientos. En consecuencia, deberían modificarse con la finalidad de corregir dichos problemas.

Las actividades en el proceso de requerimientos no se realizan simplemente en una secuencia estricta. El análisis de requerimientos continúa durante la definición y especificación, y a lo largo del proceso salen a la luz nuevos requerimientos; por lo tanto, las actividades de análisis, definición y especificación están vinculadas.



Proceso de ingeniería de requerimientos [Sommerville]

Por su parte, Pressmann [1], especifica que el espectro amplio de tareas y técnicas que llevan a entender los requerimientos se denomina *ingeniería de requerimientos*. Desde la perspectiva del proceso del software, la ingeniería de requerimientos es una de las acciones importantes de la ingeniería de software que comienza durante la actividad de comunicación y continúa en la de modelado. Debe adaptarse a las necesidades del proceso, del proyecto, del producto y de las personas que hacen el trabajo.

La ingeniería de requerimientos proporciona el mecanismo apropiado para entender lo que desea el cliente, analizar las necesidades, evaluar la factibilidad, negociar una solución razonable, especificar la solución sin ambigüedades, validar la especificación y administrar los requerimientos a medida de que se transforman en un sistema funcional. Es importante notar que algunas de estas tareas ocurren en paralelo y que todas se adaptan a las necesidades del proyecto.

En el contexto de la gestión de proyectos, los requerimientos importan por dos razones concretas: determinan el alcance de lo que hay que construir y establecen la base sobre la cual se definen los criterios de aceptación del producto. Un requerimiento mal definido al inicio se convierte inevitablemente en un conflicto de alcance durante el desarrollo.

6.1. Requerimientos funcionales y no funcionales

A menudo, los requerimientos del sistema de software se clasifican como requerimientos funcionales o requerimientos no funcionales: [3]

- *Requerimientos funcionales*: Son enunciados acerca de servicios que el sistema debe proveer, de cómo debería reaccionar el sistema a entradas particulares y de cómo debería comportarse el sistema en situaciones específicas. En algunos casos, los requerimientos funcionales también explican lo que no debe hacer el sistema. Tales requerimientos dependen del tipo de software que se esté desarrollando, de los usuarios esperados del software y del enfoque general que

adopta la organización cuando se escriben los requerimientos. Al expresarse como requerimientos del usuario, los requerimientos funcionales se describen por lo general de forma abstracta que entiendan los usuarios del sistema. Sin embargo, requerimientos funcionales más específicos del sistema detallan las funciones del sistema, sus entradas y salidas, sus excepciones, etcétera.

Ejemplos de requerimientos funcionales para el sistema MHC-PMS, que se usan para mantener información de pacientes que reciben tratamiento por problemas de salud mental:

1. Un usuario podrá buscar en todas las clínicas las listas de citas.
   ➔ El sistema debe permitir consultar información de turnos.
2. El sistema elaborará diariamente, para cada clínica, una lista de pacientes que se espera que asistan a cita ese día.
   ➔ El sistema debe generar automáticamente listados diarios.
3. Cada miembro del personal que usa el sistema debe identificarse de manera individual con su número de ocho dígitos.
   ➔ El sistema debe verificar e identificar usuarios antes de permitir el acceso.

Estos requerimientos funcionales del usuario definen las actividades específicas que debe proporcionar el sistema. Se tomaron del documento de requerimientos del usuario y muestran que los requerimientos funcionales pueden escribirse con diferentes niveles de detalle.

- *Requerimientos no funcionales*: Son limitaciones sobre servicios o funciones que ofrece el sistema. Incluyen restricciones tanto de temporización y del proceso de desarrollo, como impuestas por los estándares. Son requerimientos que no se relacionan directamente con los servicios específicos que el sistema entrega a sus usuarios. Pueden relacionarse con propiedades emergentes del sistema, como fiabilidad, tiempo de respuesta y uso de almacenamiento. Los requerimientos no funcionales, como el rendimiento, la seguridad o la disponibilidad, especifican o restringen por lo general características del sistema como un todo, más que a características o a servicios individuales del sistema. Los requerimientos no funcionales a menudo son más significativos que los requerimientos funcionales individuales. A veces se los llama requerimientos de comportamiento o desempeño. Una clasificación de este tipo de requerimientos puede observarse en la siguiente figura.

  Ejemplos de requerimientos no funcionales tomados del MHC-PMS:

  Requerimiento del producto: *"El sistema debe estar disponible el 99% del tiempo en horario laboral."*

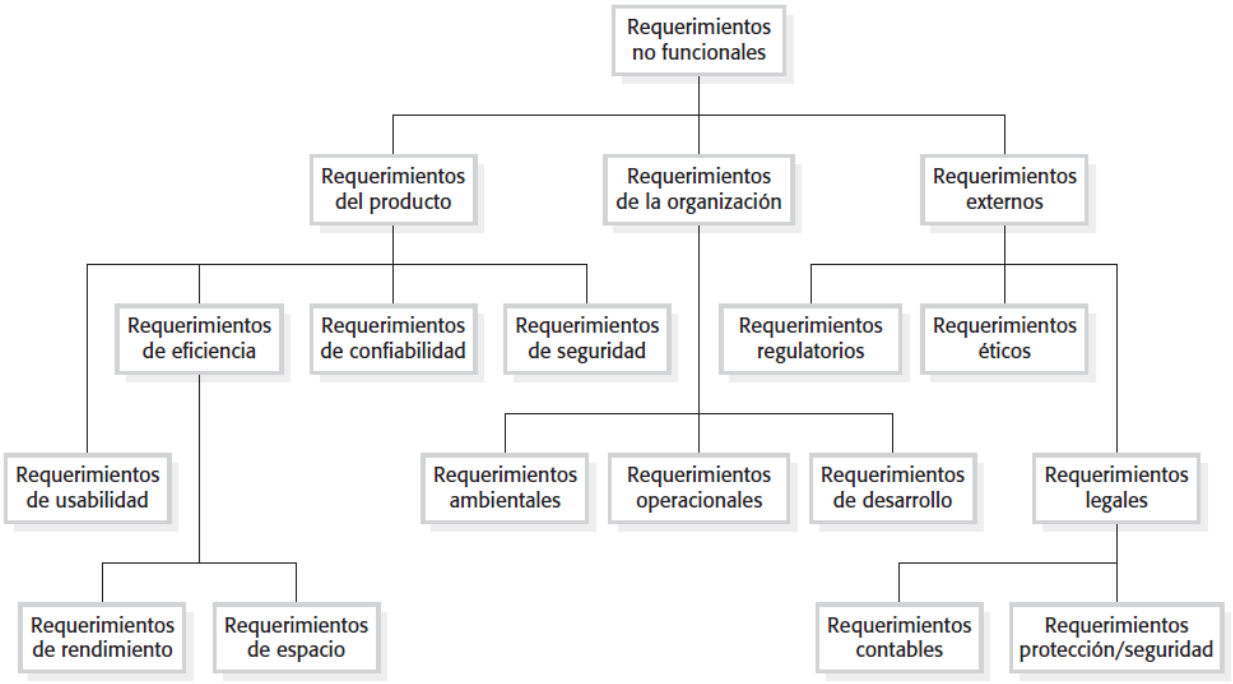
  ➔ **Regula:** rendimiento, disponibilidad, seguridad, usabilidad.

  Requerimientos de la organización: *"El sistema debe desarrollarse utilizando el stack tecnológico aprobado por el área de IT."*

  ➔ **Regula:** estándares internos, políticas, tecnologías permitidas.

  Requerimientos externos: *"El sistema debe cumplir con la Ley 25.326 de Protección de Datos Personales."*

  ➔ **Regula:** normativas legales o regulatorias que el sistema debe cumplir.



Clasificación de requerimientos no funcionales propuesta por Sommerville.

En realidad, la distinción entre los diferentes tipos de requerimientos no es tan clara como sugieren estas definiciones sencillas. Un requerimiento de un usuario interesado por la seguridad, como el enunciado que limita el acceso a usuarios autorizados, parecería un requerimiento no funcional. Sin embargo, cuando se desarrolla con más detalle, este requerimiento puede generar otros requerimientos que son evidentemente funcionales, como la necesidad de incluir facilidades de autenticación en el sistema.

Esto muestra que los requerimientos no son independientes y que un requerimiento genera o restringe normalmente otros requerimientos. Por lo tanto, los requerimientos del sistema no sólo detallan los servicios o las características que se requieren del mismo, sino también especifican la funcionalidad necesaria para asegurar que estos servicios y características se entreguen de manera adecuada.

Referencias Bibliográficas

[1] *Ingeniería de Software: Un enfoque práctico*. 7ma Edición. Roger S. Pressman. McGrawHill. 2010.

[2] *Software Engineering: a practitioner's approach*. 9th Edition. Roger S. Pressman, Bruce R. Maxim. McGrawHill. 2020.

[3] *Ingeniería de Software*. 9na Edición. Ian Sommerville. Pearson. 2011.

[4] *Software Engineering*. 10th edition. Ian Sommerville. Pearson. 2016.